CSC 351- Assignment 6
Madeline Ashton

Query 1:

```
> db.amazon.find({product_name:{$regex:"Train",$options:"i"}}).limit(5)
< {
    _id: ObjectId('6ae383e73d327ab8df55e619'),
    uniq_id: 'b17540ef7e86e461d37f3ae58b7b72ac',
    product_name: 'FunkyBuys® Large Christmas Holiday Express Festive Train Set (SI-TY1017) Toy Light / Sounds / Battery Operated & Smoke
    manufacturer: 'FunkyBuys',
    price: '£16.99',
    number_of_reviews: 2,
    number_of_answered_questions: 1,
    average_review_rating: '4.5 out of 5 stars',
    amazon_category_and_sub_category: 'Hobbies > Model Trains & Railway Sets > Rail Vehicles > Trains',
    customers_who_bought_this_item_also_bought: 'http://www.amazon.co.uk/Christmas-Holiday-Express-Festive-Train-Set-Toy/dp/B009R8S8AA |
    description: 'Size Name:Large FunkyBuys® Large Christmas Holiday Express Festive Train Set (SI-TY1017) Toy Light / Sounds / Battery C
    product_information: 'Technical Details Manufacturer recommended age:3 years and up Item model numberSI-TY1017-B    Additional Inform
    product_description: 'Size Name:Large FunkyBuys® Large Christmas Holiday Express Festive Train Set (SI-TY1017) Toy Light / Sounds / E
    items_customers_buy_after_viewing_this_item: 'http://www.amazon.co.uk/Christmas-Holiday-Express-Festive-Train-Set-Toy/dp/B009R8S8AA |
```

```
> db.amazon.countDocuments({product_name:{$regex:"Train",$options:"i"}})
< 149
```

$match $limit

HW 6 Q1 SAVE + CREATE NEW EXPORT DATA EXPORT CODE

```
[
  {
    $match:
      /**
       * query: The query in MQL.
       */
      {
        product_name: {
          $regex: "train",
          $options: "i"
        }
      }
  },
  {
    $limit:
      /**
       * Provide the number of documents to limit.
       */
      5
  }
]
```

In this query, $match and $regex are similar to the SQL “where” filtering with “like”. The $limit command in MongoDB is like the “limit” command in SQL.

Query 2:

```
> db.amazon.find({manufacturer: "Hornby"}, {product_name: 1, price: 1, _id: 0}).limit(10).sort({price:-1})
< {
    product_name: 'Hornby Railroad 00 Gauge LNER Class A1 Flying Scotsman with TTS Sound Steam Locomotive',
    price: '£98.50'
  }
  {
    product_name: 'Hornby The Western Spirit OO Gauge Electric Train Set',
    price: '£94.46'
  }
  {
    product_name: 'Hornby 00 Gauge 152mm GNR 0-6-2T Steam Locomotive Model',
    price: '£89.99'
  }
  {
    product_name: 'Hornby 00 Gauge 152mm BR 0-6-2T Steam Locomotive Model',
    price: '£89.99'
  }
  {
    product_name: 'Hornby Gloucester City Pullman Train Set',
    price: '£82.31'
  }
```



In this query, the “limit” command translates to $limit, and the “order by” command translates to $sort. The “where” clause becomes a $match command, and the $project command is like SQL’s initial selection.

Query 3:

```
db.amazon.countDocuments({$or:[{manufacturer:"Hornby"},{manufacturer:"Bachmann"}],average_review_rating:{$regex:"4.9"}})
2
```

```
[
  {
    $match:
      /**
       * query: The query in MQL.
       */
      {
        $or: [
          {
            manufacturer: "Hornby"
          },
          {
            manufacturer: "Bachmann"
          }
        ]
      }
  },
  {
    $match:
      /**
       * query: The query in MQL.
       */
      {
        average_review_rating: {
          $regex: "4.9"
        }
      }
  },
  {
  },
  {
    $count:
      /**
       * Provide the field name for the count.
       */
      "string"
  }
]
```

PIPELINE OUTPUT PREVIEW

Sample of 1 document

```
string: 2
```

In this query, once again, “where” filtering is translated to $match and $regex. The “or’ statement works similarly to SQL, while the “and” statement is replaced by a double input to $match.

Query 4:

```
> db.amazon.aggregate([{$group: {_id: "$manufacturer", total_prod_count: {$sum:1}}}, {$sort:{total_prod_count: -1}}])
< {
    _id: 'LEGO',
    total_prod_count: 171
  }
  {
    _id: 'Disney',
    total_prod_count: 167
  }
  {
    _id: 'Oxford Diecast',
```

```
[
  {
    $group:
      /**
       * _id: The id of the group.
       * fieldN: The first field name.
       */
      {
        _id: "$manufacturer",
        total_product_count: {
          $sum: 1
        }
      }
  },
  {
    $sort:
      /**
       * Provide any number of field/order pairs.
       */
      {
        total_product_count: -1
      }
  }
]
```

PIPELINE OUTPUT PREVIEW

OUTPUT OPTI

Sample of 10 documents

```
_id: "LEGO"
total_product_count: 171

_id: "Disney"
total_product_count: 167

_id: "Oxford Diecast"
total_product_count: 156

_id: "Playmobil"
total_product_count: 147

_id: "Star Wars"
total_product_count: 120

_id: "Mattel"
total_product_count: 114
```

The “group by” SQL command was replaced by the $group function. The “count” was performed by the $sum accumulator, and the ordering was done by the $sort command.